

PROGRAMAÇÃO ORIENTADA A OBJETOS - CHECKPOINT 1

Nome: _____

Nota: _____

RESPOSTAS

QUESTÃO 1	a	b	c	d	e
QUESTÃO 2	a	b	c	d	e
QUESTÕES 3-7	3	4	5	6	7
QUESTÕES 8-12	8	9	10	11	12

CENÁRIO: ELEVADOR

Os arquivos `Elevador.java` e `Main.java` estão no mesmo pacote. Considere exatamente o código abaixo.

Elevador.java	Main.java
<pre> 1 public class Elevador { 2 int andarAtual = 0; 3 boolean portaAberta = false; 4 5 void subir() { 6 andarAtual = andarAtual + 1; 7 } 8 void abrirPorta() { 9 portaAberta = true; 10 } 11 void fecharPorta() { 12 portaAberta = false; 13 } 14 int getAndarAtual() { 15 return andarAtual; 16 } 17 boolean getPortaAberta() { 18 return portaAberta; 19 } 20 }</pre>	<pre> 1 public class Main { 2 public static void main(String[] args) { 3 Elevador principal = new Elevador(); 4 Elevador painel = principal; 5 Elevador servico = new Elevador(); 6 7 principal.subir(); 8 painel.abrirPorta(); 9 10 System.out.println(principal.getAndarAtual()); 11 System.out.println(principal.getPortaAberta()); 12 13 painel.fecharPorta(); 14 painel.subir(); 15 16 System.out.println(painel.getAndarAtual()); 17 System.out.println(painel.getPortaAberta()); 18 19 servico.subir(); 20 servico.abrirPorta(); 21 22 System.out.println(servico.getAndarAtual()); 23 } 24 }</pre>

QUESTÕES

Questão 1 (25 pontos)

Faça o teste de mesa e indique o que será impresso em cada linha.

- | | |
|-------------|-------------|
| a) Linha 10 | d) Linha 17 |
| b) Linha 11 | e) Linha 22 |
| c) Linha 16 | |

Questão 2 (25 pontos)

Considere a evolução da classe e marque cada afirmação como V ou F.

Os getters e os métodos `abrirPorta()` e `fecharPorta()` permanecem inalterados.

Trechos alterados em Elevador.java
<pre> private int andarAtual; private boolean portaAberta; void subir() { if (portaAberta == false) { if (andarAtual < 3) { andarAtual = andarAtual + 1; } } }</pre>

- a) Em Main, a instrução `principal.andarAtual = 2;` compila.
- b) O método `subir()` pode alterar `andarAtual`, embora o campo seja privado.

- c) Do térreo, após `abrirPorta()` e `subir()`, o andar continua em 0.
- d) Com a porta fechada e o andar em 3, chamar `subir()` leva ao andar 4.
- e) Do térreo, após `abrirPorta()`, `fecharPorta()` e `subir()`, o andar continua em 0.

Questão 3 (5 pontos)

O programa precisa guardar o andar e a situação da porta e executar as ações de subir, abrir e fechar. Qual organização representa melhor um elevador?

- A. Manter os dois dados em `Main` e repetir ali todas as ações.
- B. Criar uma classe apenas com o andar e a porta e programar todas as ações em `Main`.
- C. Criar `Elevador` com o andar, a porta e os métodos que realizam essas ações.
- D. Manter os dados em variáveis de `Main` e passar todos eles a cada método externo.
- E. Criar uma classe para o andar, outra para a porta e realizar as ações em `Main`.

Questão 4 (5 pontos)

Ao executar `Elevador principal = new Elevador();`, o que cada parte da instrução representa?

- A. `Elevador` é a variável; `principal` é a classe; `new` copia um objeto.
- B. `Elevador` é a classe; `principal` guarda uma referência para o novo objeto.
- C. `principal` é o objeto; `Elevador` guarda a referência usada por `new`.
- D. `new Elevador()` é a variável; `principal` declara outra classe.
- E. Os três nomes representam o mesmo papel e identificam diretamente o objeto.

Questão 5 (5 pontos)

Quando `abrirPorta()` é executado, o valor de `portaAberta` muda. Qual alternativa descreve corretamente esses dois membros?

- A. `portaAberta` é estado; `abrirPorta()` é comportamento.
- B. `portaAberta` é comportamento; `abrirPorta()` é estado.
- C. Ambos são estados porque pertencem ao mesmo objeto.
- D. Ambos são comportamentos porque podem ser acessados com ponto.
- E. O campo é uma classe e o método é um objeto.

Questão 6 (5 pontos)

Queremos um método que receba um andar de destino inteiro e não retorne valor. Qual assinatura atende ao requisito?

- A. `int irPara()`
- B. `void irPara()`
- C. `int irPara(int destino)`
- D. `void irPara(int destino)`
- E. `void irPara(float portaAberta)`

Questão 7 (5 pontos)

O código compila? Se não, qual linha impede a compilação?

```
Trecho em Elevador.java
1 int andarAtual;
2 void mostrar() {
3     int anterior;
4     System.out.println(andarAtual);
5     System.out.println(anterior);
6 }
```

- A. Compila e as linhas 4 e 5 imprimem 0.
- B. Não compila: a linha 5 tenta ler a variável local `anterior` sem inicializá-la.
- C. Não compila: a linha 4 tenta ler o campo `andarAtual` sem inicializá-lo.
- D. Compila, mas a execução falha na linha 5 ao acessar `anterior`.
- E. Não compila: apenas declarar a variável local na linha 3 já é inválido.

Questão 8 (5 pontos)

O elevador suporta no máximo 8 pessoas. Ao embarcar um grupo, a lotação não pode ultrapassar esse limite. Qual solução mantém essa regra em um único responsável?

- A. Cada trecho de `Main` consulta a lotação e decide se soma o grupo.
- B. Um auxiliar de `Main` calcula a lotação e altera o campo.
- C. `Main` verifica o grupo; outro método externo verifica o limite.
- D. Um setter recebe a lotação calculada pelo código externo.
- E. `Elevador.embarcar(quantidade)` consulta sua lotação e preserva o limite.

Questão 9 (5 pontos)

Considere o método e a chamada abaixo. O que acontece?

Trecho em `Elevador.java` e `Main.java`

```
1 int calcularDiferenca(int destino) {
2     return destino - andarAtual;
3 }
4 int diferenca = elevador.calcularDiferenca(3);
```

Considere que `andarAtual` vale 1 antes da chamada.

- A. `diferenca` recebe 3 e `andarAtual` passa a ser 3.
- B. `diferenca` recebe 2 e `andarAtual` passa a ser 3.
- C. O código não compila porque um método com retorno `int` não pode receber parâmetro.
- D. `diferenca` recebe 2 e `andarAtual` continua em 1.
- E. O método não retorna valor porque não foi declarado com `void`.

Questão 10 (5 pontos)

Dois elevadores são criados separadamente e começam no térreo com a porta fechada. Depois, apenas `a.subir()` é chamado. Qual afirmação está correta?

- A. Antes da chamada, `a == b` é `true`; depois, ambos ficam no andar 1.
- B. Antes da chamada, os estados diferem; depois, somente `b` fica no andar 1.
- C. Antes da chamada, os estados são iguais e `a == b` é `false`; depois, somente `a` fica no andar 1.
- D. Antes da chamada, `a == b` é `true`; depois, somente `a` fica no andar 1.
- E. Antes da chamada, os estados são iguais; por isso, alterar `a` também altera `b`.

Questão 11 (5 pontos)

`getAndarAtual()` não possui modificador de acesso. Como `Main` e `Elevador` estão no mesmo pacote, qual afirmação está correta?

- A. `Main` pode chamá-lo: o acesso de pacote permite uso no mesmo pacote.
- B. `Main` não pode chamá-lo porque o método não é `public`.
- C. Ele só pode ser chamado se `andarAtual` também for público.
- D. Ele se torna privado automaticamente por não declarar `public`.
- E. Ele altera o pacote de `Main` durante a execução.

Questão 12 (5 pontos)

O elevador só pode ficar entre os andares 0 e 3. Analise a alteração abaixo. Qual diagnóstico está correto?

Trecho em `Elevador.java`

```
1 private int andarAtual;
2 public void definirAndar(int novoAndar) {
3     andarAtual = novoAndar;
4 }
```

- A. O campo `private` garante sozinho que todo andar seja válido.
- B. Receber um `int` impede automaticamente andares negativos.
- C. Nem os métodos da própria classe podem alterar o campo.
- D. O acesso direto fechou, mas o método ainda aceita qualquer andar.
- E. Um método público não pode alterar um campo privado.